Runbook: Testing the Firebase-Distributed Lava Client Against the Thinker Network API

Date: 2026-06-23
Applies to: Client 1.3.11-1071 (debug `.client.dev` / release `.client`), api-app 0.2.11-17
Purpose: Step-by-step procedure for an operator to verify that the Firebase-distributed Lava Android client can reach the live `lava-api-go` stack running on the "thinker" development host, and to read §6.AC search telemetry in Firebase Crashlytics to confirm or rule out the H1 auth-header-overwrite bug.

---

1. How the Client Reaches an API: Two Paths

The client supports two distinct ways to talk to a Lava API service. Both are discovered through the onboarding `ApiSelectionStep` screen (`feature/onboarding/src/main/kotlin/lava/onboarding/steps/ApiSelectionStep.kt`).

Path A — ON-DEVICE api-app engine (loopback)

The separately-installed `digital.vasic.lava.api` app runs `lava-api-go` as an Android background service on `127.0.0.1`. It exposes its per-install handoff key via a `ContentProvider`, which the client reads through `ApiKeyStore` and injects as the `Lava-Auth` header value **before** `AuthInterceptor` runs. The on-device engine advertises itself over mDNS as service type `_lava-api._tcp` with TXT record `platform=android`. When the `ApiSelectionStep` ViewModel scans via `LocalNetworkDiscoveryService` (Android `NsdManager`) it resolves this to `Endpoint.GoApi(host="127.0.0.1", port=<dynamic>)` with `platform="android"`. The subtitle rendered by `displaySubtitle()` reads "Lava API · Android device".

Path B — NETWORK-EXPOSED lava-api-go on thinker (LAN HTTPS)

The thinker host runs `lava-api-go` in `docker-compose.yml` with `network_mode: host`. It advertises two services:

- **Prod stack:** `_lava-api._tcp` on port 8443 — service instance name does NOT carry `platform=android`
- **Dev stack:** `_lava-api-dev._tcp` on port 8543 — likewise

When `NsdManager` discovers the thinker's `_lava-api._tcp` record, `LocalNetworkDiscoveryService` resolves it to `Endpoint.GoApi(host="192.168.0.228", port=8443)`. The TXT record's `engine=go` attribute is used; because `platform` is absent (or `platform=server`), `displaySubtitle()` renders "Lava API · On this network". The operator taps this entry and the ViewModel runs an HTTPS `/health` connectivity probe; on `{"status":"alive"}` it persists the endpoint and advances onboarding.

A third section on the `ApiSelectionStep` screen ("Cloud / remote server") allows typing a custom `https://host:port` URL directly — useful when mDNS broadcast is not reachable across subnets.

Key difference for auth: Path A uses a per-install handoff key (Mechanism B). Path B uses the build-time UUID embedded in the APK at `LAVA_AUTH_UUID` (Mechanism A, injected by `AuthInterceptor`). The H1 hypothesis is that `AuthInterceptor` overwrites the Mechanism B key on Path A because both use the same `Lava-Auth` header field name. The Crashlytics `base_url_host` key distinguishes which path is actually in use when a 401 fires (see §4 below).

---

2. Self-Signed Cert Handling: No On-Device Trust Required

The thinker's `lava-api-go` stack uses a self-signed RSA-2048 certificate with SAN `IP:192.168.0.228`, valid to 2027-06-23 (generated by `lava-api-go/scripts/gen-cert.sh`).

The operator does NOT need to install this certificate on the Android device.

The Lava client routes all `Endpoint.GoApi` traffic through the `@Named("lan") OkHttpClient` binding defined in `core/network/impl/src/main/kotlin/lava/network/di/NetworkModule.kt` (lines 149–180):

```
@Provides @Singleton @Named("lan")
fun lanOkHttpClient(...): OkHttpClient {
    val permissiveTrustManager = object : X509TrustManager {
        override fun checkServerTrusted(chain:
        Array<X509Certificate>, authType: String) {
            // No-op: trust any LAN-side server cert
        }
        ...
    }
    ...
    .sslSocketFactory(sslContext.socketFactory,
        permissiveTrustManager)
    .hostnameVerifier { _, _ -> true }
    .build()
}
```

This client bypasses Android's `X509TrustManager` chain validation entirely — `checkServerTrusted` is a no-op. It also sets `hostnameVerifier { _, _ -> true }`, so the IP-SAN in the cert need not match the IP the client connects to. TLS encryption is still active (TLS 1.3); the relaxation is authentication only. This is the same threat model the codebase already uses for cleartext on LAN mirrors.

`app/src/main/res/xml/network_security_config.xml` line 37 explicitly documents this:

> "The `lava-api-go` HTTPS path on port 8443 does NOT depend on this section — that path is wired through the LAN-permissive OkHttpClient and bypasses NSC's trust-anchor resolution entirely."

Android's `network_security_config.xml` only controls cleartext HTTP for the legacy Ktor proxy path on port 8080. It has no effect on HTTPS port 8443.

---

3. Test Procedure: Firebase Install → Onboard → Search

3.1 Prerequisites

- Thinker prod stack healthy (verify before starting):

```
curl -k https://192.168.0.228:8443/health   # →
        {"status":"alive"}
curl -k https://192.168.0.228:8443/ready    # →
        {"status":"ready"}
```

- Android device on the same Wi-Fi subnet as thinker (`192.168.0.x`).

- Firebase App Distribution invite accepted; APK installed (`digital.vasic.lava.client.dev` for debug).

3.2 Fresh-install onboarding

1. **Launch the app.** The `ApiSelectionStep` screen appears: "Searching for APIs on your network…" (spinner + `api-discovery-searching` content-description node visible).
2. **Wait for discovery** (up to ~10 s). The thinker's `_lava-api._tcp` advertisement should resolve. Expected: "Found 1 API:" + a row showing `192.168.0.228:8443` / "Lava API · On this network".
3. **Tap the thinker row.** The row enters probe state (spinner on row). The ViewModel calls `ConnectionService.isReachable` against `https://192.168.0.228:8443/health`.
4. **On success:** Onboarding advances to the provider-selection step. On failure: "Could not reach this API: " appears. See §5 for thinker controls.
5. **Select at least one provider** (e.g. RuTracker or Rutor) and complete onboarding.
6. **Run a search.** Enter any query and submit.

3.3 Expected outcomes

Scenario	Expected result
Thinker reachable, auth correct	Search results appear
Thinker reachable, 401 returned	No results; §6.AC non-fatal fires in Crashlytics
mDNS not resolved	"No APIs discovered" — use Cloud/remote-server section to type `https://192.168.0.228:8443`
Thinker unreachable	Probe fails at onboarding; fix thinker stack (§5) before retrying

3.4 Auth status quick-check (adb)

While the device is connected via USB:

```
adb logcat -s LavaAuth:D ApiBackedTrackerClient:D
        SwitchingNetworkApi:D
```

Look for `401` responses and whether the logged endpoint host is `192.168.0.228` (thinker) or `127.0.0.1` (on-device engine). This confirms which auth path is active before checking Crashlytics.

---

4. Reading §6.AC Search Telemetry in Firebase Crashlytics

The client ships §6.AC comprehensive non-fatal telemetry. Every search HTTP failure calls `SearchResultViewModel.recordProviderFailure()` (line 175 of `docs/telemetry/2026-06-23-comprehensive-nonfatal-telemetry.md`), which enriches the `ApiHttpException` context and forwards it to `AnalyticsTracker.recordNonFatal()` → `FirebaseCrashlytics.recordException()`.

4.1 Crashlytics custom keys to inspect

Open the Firebase Console → Crashlytics → Non-fatals filter: `ApiHttpException`.

Key	What it tells you
`http_status`	HTTP response code. `401` = auth rejected by the server
`request_url`	Query-stripped URL path (e.g. `/api/rutracker/search`). Note: query params are redacted by `stripQueryForTelemetry()` (§6.H)
`http_method`	`GET` or `POST`
`base_url_host`	**The host the client connected to.** `192.168.0.228` = thinker; `127.0.0.1` = on-device engine; `api.vasic.digital` = cloud

4.2 Diagnosing the H1 hypothesis

H1 (HIGHEST priority): `AuthInterceptor` at `core/network/impl/src/main/kotlin/lava/network/impl/AuthInterceptor.kt` line 63 calls `request.newBuilder().header(fieldName, headerValue)` — `header()` (not `addHeader()`) **replaces** any existing value for that field. If the routing layer already set the per-install Mechanism B key from `ApiKeyStore` via `withAuth()`, `AuthInterceptor` overwrites it with the build-time UUID, causing the server's HMAC verification to fail with 401.

Crashlytics filter combination to confirm H1:

1. `http_status = 401`
2. `base_url_host = 127.0.0.1` — confirms the 401 is hitting the ON-DEVICE engine (which uses Mechanism B keys), NOT the thinker (which uses Mechanism A UUID)

If you see 401s with `base_url_host = 127.0.0.1`, H1 is confirmed: the on-device engine is rejecting the UUID because its `AuthActiveClients` list contains only the per-install key, not the build-time UUID.

If 401s show `base_url_host = 192.168.0.228`, the thinker stack is rejecting the UUID — investigate `lava-api-go/internal/auth/middleware.go` and whether `LAVA_AUTH_UUID` in the APK matches the value in `lava-api-go`'s `.env`.

Note: Firebase Crashlytics has no server-side ingest API. The Go service (`lava-api-go`) emits errors to Loki/Grafana (OTLP pipeline) — not to Crashlytics. Server-side 401 causes will appear in the thinker's container logs (`docker logs lava-api-go`), not in the Firebase Console.

4.3 Crashlytics filter steps

1. Firebase Console → your project → Crashlytics.
2. Switch to "Non-fatals" tab.
3. Filter by issue title containing `ApiHttpException`.
4. Open an event → scroll to "Keys" section.
5. Check `http_status`, `base_url_host`, `request_url` as described above.

---

5. Thinker Stack Controls

LAN URLs

Stack	Base URL	mDNS type
Prod	`https://192.168.0.228:8443`	`_lava-api._tcp`
Dev	`https://192.168.0.228:8543`	`_lava-api-dev._tcp`

Health check commands (run on thinker)

```
# Quick sanity — no auth required
curl -k https://192.168.0.228:8443/health   # {"status":"alive"}
curl -k https://192.168.0.228:8443/ready    # {"status":"ready"}

# Container status
docker ps --filter name=lava-api-go --format "table {{.Names}}
        \t{{.Status}}"

# Tail API logs (shows auth middleware decisions)
docker logs -f lava-api-go
```

```
# Restart stack
./stop.sh && ./start.sh
```

mDNS advertisement verification (on device or LAN host)

```
# On the Android device via adb (requires network-tools or mdns-
        scan on host)
adb shell "getprop | grep nsd"

# On the thinker host (if avahi-browse available)
avahi-browse -r _lava-api._tcp
```

The `docker-compose.yml` uses `network_mode: host` so JmDNS broadcasts reach the LAN directly; no port-mapping is needed for mDNS to work.

Cert details

- Algorithm: RSA-2048
- SAN: `IP:192.168.0.228`
- Valid to: 2027-06-23
- Generated by: `lava-api-go/scripts/gen-cert.sh`
- No client-side installation required (LAN OkHttpClient bypasses validation — see §2)

Source Citations (File:Line)

1. **Cert handling (LAN permissive OkHttpClient):** `core/network/impl/src/main/kotlin/lava/network/di/NetworkModule.kt:156–179` — `checkServerTrusted` is a no-op; `hostnameVerifier { _, _ -> true }`. The KDoc at lines 116–148 explicitly states the SP-3.1 rationale: self-signed LAN cert without device-side CA install.

2. **NSC does not govern port 8443:** `app/src/main/res/xml/network_security_config.xml:37` — Comment: "The `lava-api-go` HTTPS path on port 8443 does NOT depend on this section — that path is wired through the LAN-permissive OkHttpClient and bypasses NSC's trust-anchor resolution entirely."

3. **mDNS discovery → Endpoint.GoApi resolution + displaySubtitle platform discriminator:** `feature/onboarding/src/main/kotlin/lava/onboarding/steps/ApiSelectionStep.kt:337–346` — `displaySubtitle()` returns `"Lava API · ${discoveredApiLabel(platform)}"` for `Endpoint.GoApi`; `platform="android"` distinguishes the on-device api-app from the thinker network instance.